

PORTADA

Calidad de Software

Semana 3 — Historias de usuario, criterios de aceptación y el plan de aseguramiento

Universidad de Antioquia · Ingeniería de Sistemas



REPASO RELÁMPAGO

Lo que ya tenemos de la semana 2

- **QA** es preventivo y mira el proceso; **QC** es detectivo y mira el producto.
- **Verificación:** ¿lo construimos correctamente? **Validación:** ¿construimos lo correcto?
- El **modelo en V** empareja cada fase de desarrollo con su fase de prueba, desde el inicio.
- La calidad es **responsabilidad de todo el equipo**, no solo de un rol dedicado.



TRANSICIÓN

Hoy: bajamos del proceso al artefacto

El modelo en V pone **pruebas de aceptación** frente a los **requisitos de usuario**. Hasta ahora hemos hablado de esa pareja en abstracto — hoy vemos con qué artefacto concreto se escribe cada lado.

Ese artefacto es la **historia de usuario**, y el lenguaje para probarla es **Gherkin**.



CONCEPTO

Qué es una historia de usuario

Es una **descripción breve e informal** de una funcionalidad, escrita desde la perspectiva de quien la va a usar — no una especificación técnica exhaustiva. Formato clásico:

Como <tipo de usuario>, **quiero** <una funcionalidad>, **para** <un beneficio o razón>.

Ejemplo: *"Como cliente de la clínica, quiero recibir un recordatorio por correo un día antes de mi turno, para no olvidarlo."*



ORIGEN

Kent Beck, Ron Jeffries y la Extreme Programming

Las historias de usuario nacieron a finales de los 90 dentro de **Extreme Programming (XP)**, la metodología ágil que Kent Beck lideró en el proyecto C3 de Chrysler. Ron Jeffries, coautor de XP, buscaba una forma de planear que no dependiera de documentos de requisitos extensos y difíciles de cambiar.

La idea: tarjetas físicas, cortas a propósito, que sirvieran de **recordatorio para una conversación** —no como contrato cerrado — entre el equipo y quien pide la funcionalidad.

Jeffries resumió la idea en tres palabras que siguen citándose: "**Card, Conversation, Confirmation**" — la tarjeta, la conversación que provoca, y la confirmación (los criterios de aceptación) de que quedó bien entendida.

CONCEPTO

INVEST: seis criterios de una buena historia

En 2003, Bill Wake propuso un acrónimo para revisar si una historia está bien escrita antes de aceptarla en el plan de trabajo:

- **Independiente:** se puede planear y entregar sin depender de otra historia.
- **Negociable:** es punto de partida para conversar, no un contrato cerrado.
- **Valiosa:** entrega valor claro a quien la pidió.
- **Estimable:** el equipo puede darle un tamaño aproximado.
- **Pequeña:** cabe dentro de una sola iteración.
- **Testeable:** existe una forma clara de comprobar que quedó hecha.

CONCEPTO

Qué es un criterio de aceptación

Es la respuesta a la "T" de INVEST: la **condición concreta y verificable** que el equipo y quien pidió la historia acuerdan como definición de "hecho". No es la historia misma, es lo que la hace **testable**.

- Una historia normalmente tiene **varios** criterios de aceptación.
- Se escriben **antes** de programar, no después — son parte del acuerdo, no una ocurrencia tardía.
- Si no se puede escribir un criterio de aceptación claro, probablemente la historia no está lista.



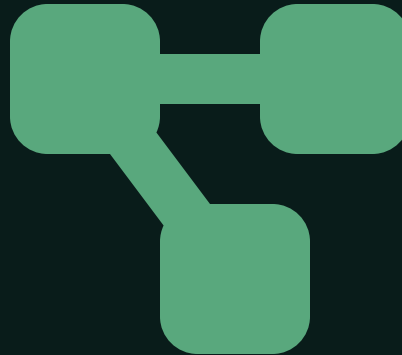
CONCEPTO

De la historia a la prueba

Aquí se cierra el emparejamiento del modelo en V que vimos la semana pasada:

Historia de usuario (el requisito) → **criterios de aceptación** (la condición de éxito) → **prueba de aceptación** (la comprobación real, con el sistema en ejecución).

Cada criterio de aceptación es, en potencia, una prueba de aceptación ya redactada — solo falta ejecutarla.



TRANSICIÓN

Falta un lenguaje común

"El sistema debe validar bien las fechas" es un criterio de aceptación mal escrito: ambiguo, no ejecutable, interpretable de mil formas distintas por distintas personas.

Hace falta una estructura fija para escribir criterios de aceptación que cualquiera —negocio, desarrollo, pruebas— lea igual.



CONCEPTO · ORIGEN

Qué es Gherkin

Gherkin es un lenguaje estructurado, en texto plano y en el idioma de quien lo escribe, para redactar criterios de aceptación como escenarios legibles por personas **y** ejecutables por una herramienta.

Nació junto con **Cucumber**, la herramienta que Aslak Hellesøy creó en 2008 para conectar esos escenarios directamente con pruebas automatizadas — de ahí que ambos compartan nombre de vegetal.



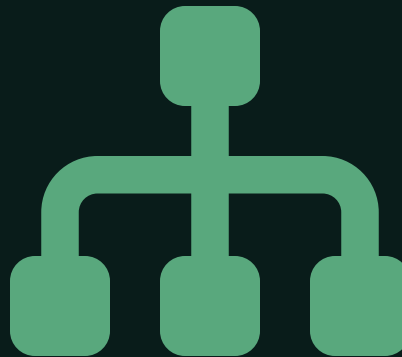
CONCEPTO

La estructura Given-When-Then

En español, **Dado-Cuando-Entonces**. Todo escenario en Gherkin se arma con tres partes:

- **Dado** (*Given*) — el contexto inicial, cómo empieza el mundo antes de actuar.
- **Cuando** (*When*) — la acción concreta que dispara el comportamiento.
- **Entonces** (*Then*) — el resultado esperado, lo que se debe poder comprobar.

Se pueden encadenar pasos con **Y** (*And*) o **Pero** (*But*) cuando un bloque necesita más de una condición.



EJEMPLO

Un escenario en Gherkin

Para la historia de recordatorio de turno de la diapositiva 4:

```
Feature: Recordatorio de turno por correo
```

```
  Scenario: Enviar recordatorio un día antes
```

```
    Given un cliente tiene un turno agendado para mañana
```

```
    And el cliente tiene un correo válido registrado
```

```
    When el sistema ejecuta el proceso diario de recordatorios
```

```
    Then el cliente recibe un correo con la fecha y hora del turno
```

Cada Scenario es, literalmente, un criterio de aceptación ejecutable.

CONCEPTO

Gherkin dentro de BDD

Gherkin es la notación; la práctica que lo envuelve se llama **BDD** (*Behavior-Driven Development, desarrollo guiado por comportamiento*), propuesta por Dan North en 2003 como una evolución del desarrollo guiado por pruebas (TDD).

La idea de BDD: describir el comportamiento esperado **en el lenguaje del negocio** antes de escribir código, para que negocio, desarrollo y pruebas discutan sobre el mismo texto — no sobre interpretaciones distintas de un documento técnico.



CONTRAEJEMPLO

Una historia mal escrita

Mal: *"Como usuario, quiero que el sistema sea rápido y fácil de usar."*

- No dice quién es el usuario ni qué acción concreta realiza.
- "Rápido" y "fácil" no son testeables sin un criterio de aceptación que los aterrice.
- Incumple varias letras de INVEST: no es pequeña, no es estimable, no es testeable.

Mejor: *"Como recepcionista, quiero agendar un turno en menos de tres clics, para atender la fila sin hacer esperar al siguiente cliente."* — con un criterio de aceptación que mida esos tres clics.

INTERACCIÓN

Ejercicio rápido: escriban un Dado-Cuando-Entonces

5 minutos — en el chat o en voz alta:

Historia: *"Como cliente, quiero cancelar un turno con más de 2 horas de anticipación, para liberar el espacio a otro cliente."*

Escriban un escenario Gherkin completo (Dado / Cuando / Entonces) que sirva como criterio de aceptación de esa historia.

Este ejercicio es el calentamiento del taller de hoy.

TRANSICIÓN

Ahora: quién escribe esto

Historias y criterios no los escribe una sola persona en aislamiento — igual que la responsabilidad compartida de QA que vimos la semana pasada.



CONCEPTO

Quién escribe las historias y sus criterios

- **Product Owner** (o quien haga ese papel): propone la historia y su "para qué" — el valor de negocio.
- **El equipo de desarrollo:** aporta viabilidad técnica y ayuda a partir historias grandes en pequeñas.
- **QA / testers:** empujan los criterios de aceptación hacia lo testeable — suelen ver los casos límite que a los demás se les escapan.

La conversación de las "tres Cs" de Jeffries es literalmente esta reunión de tres perspectivas.



TRANSICIÓN

Del criterio a la política

Las historias y sus criterios funcionan a nivel de una funcionalidad. Pero, ¿cómo se asegura la calidad a nivel de **todo el proyecto**, de principio a fin? Con un documento que ya se mencionó hace dos semanas y nunca se explicó: el **SQAP**.



CONCEPTO

Qué es un SQAP (IEEE-730)

SQAP — *Software Quality Assurance Plan*, Plan de Aseguramiento de la Calidad del Software — es el documento que describe, para un proyecto específico, **cómo se va a aplicar el aseguramiento de calidad (QA)** que vimos la semana pasada: qué estándares se siguen, quién revisa qué, y con qué frecuencia.

El estándar que define su estructura es **IEEE-730**, publicado originalmente en 1979 —el mismo año del artículo de Boehm— y actualizado varias veces desde entonces.



CONCEPTO

Qué contiene un SQAP

Propósito y alcance	Qué cubre el plan y a qué proyecto aplica
Documentación	Qué documentos deben existir y quién los aprueba
Estándares y convenciones	Normas de código, nomenclatura, revisiones de diseño
Revisiones y auditorías	Cuándo y cómo se revisa el trabajo
Pruebas	Qué se prueba, con qué técnicas, en qué niveles
Reporte de problemas	Cómo se registra y da seguimiento a un defecto
Control de código y medios	Versionamiento, backups, gestión de configuración

CONCEPTO EN DISPUTA

¿Vale la pena un SQAP completo en un equipo ágil pequeño?

IEEE-730 nació pensando en proyectos grandes, con contratos formales y entregas por fases. Un equipo de cuatro personas en un sprint de dos semanas difícilmente va a escribir un documento de quince secciones antes de programar.

La postura pragmática, no la del estándar al pie de la letra: usar la **lista de temas** de IEEE-730 como checklist de decisiones que el equipo debe tomar y dejar por escrito en algún lugar —un README, un wiki— aunque no exista un documento formal llamado "SQAP".

El riesgo real no es "no tener un PDF llamado SQAP": es no haber decidido, para el proyecto, cómo se van a revisar, probar y versionar las cosas.

APLICADO

De vuelta a Knight Capital

Con el vocabulario de hoy, el caso de la semana 1 se puede leer como un SQAP con huecos concretos:

- **Sin control de código:** nada impedía que código muerto de 2003 siguiera en producción nueve años después.
- **Sin revisión obligatoria:** el despliegue de ocho servidores lo hizo una sola persona, sin segunda revisión.
- **Sin reporte de problemas funcional:** las 97 alertas existían, pero nadie las trataba como reporte de problemas.

No es que a Knight le faltara buena voluntad: le faltaba un plan escrito que alguien pudiera auditar.



SÍNTESIS

Lo que hay que llevarse de hoy

1. Una historia de usuario es un **recordatorio para una conversación**, no un contrato cerrado.
2. **INVEST** es el checklist para saber si una historia está lista.
3. Los **criterios de aceptación** son lo que vuelve una historia testeable — se escriben antes de programar.
4. **Gherkin** (Dado-Cuando-Entonces) da un lenguaje único para escribirlos, legible y ejecutable.
5. El **SQAP** (IEEE-730) hace a nivel de proyecto lo que los criterios hacen a nivel de historia: dejar por escrito cómo se asegura la calidad.



INSTRUCCIONES

Taller de hoy: historias de usuario en Gherkin

2 horas, en sala de Zoom asignada — grupo aleatorio de la semana

El docente asigna un dominio distinto a cada equipo (por ejemplo: una app de domicilios, un sistema de biblioteca, una plataforma de citas médicas). Cada equipo debe entregar:

- **5 historias de usuario** en el formato "Como... quiero... para...", que cumplan INVEST.
- Al menos **un criterio de aceptación en Gherkin** (Dado-Cuando-Entonces) por cada historia.

La entrega cierra al terminar las dos horas. Recuerden: al enviar, cada integrante reporta en privado quién trabajó.



CIERRE

Para la próxima semana

Antes de la clase de la semana 4, hay que llegar con esto leído:

- Referentes básicos: ISO 9000, ISO/IEC 25000 (SQuaRE), CMMI, IEEE-730.
- Evaluaciones, revisiones y auditorías de calidad.

Empieza la **Unidad 2**, comprimida a dos semanas. Los grupos siguen siendo **aleatorios** hasta la semana 5. El taller de la semana 4: acta de walkthrough sobre un fragmento de código con defectos plantados.

